
Objective: Practice RISC-V Assembly Language

In this lab, you will write and debug RISC-V assembly programs using the RARS assembler and simulator. RARS supports the base machine instructions together with *pseudo-instructions* and assembler directives. Because assembler syntax and supported directives can vary, use the pinned RARS version provided by the course Docker image.

.....

1 Open the Lab Files

Complete the setup and course-update steps in the course repository README before starting. Open a terminal inside the course container and change to:

```
cd /workspace/labs/lab2
ls
```

.....

2 Start RARS

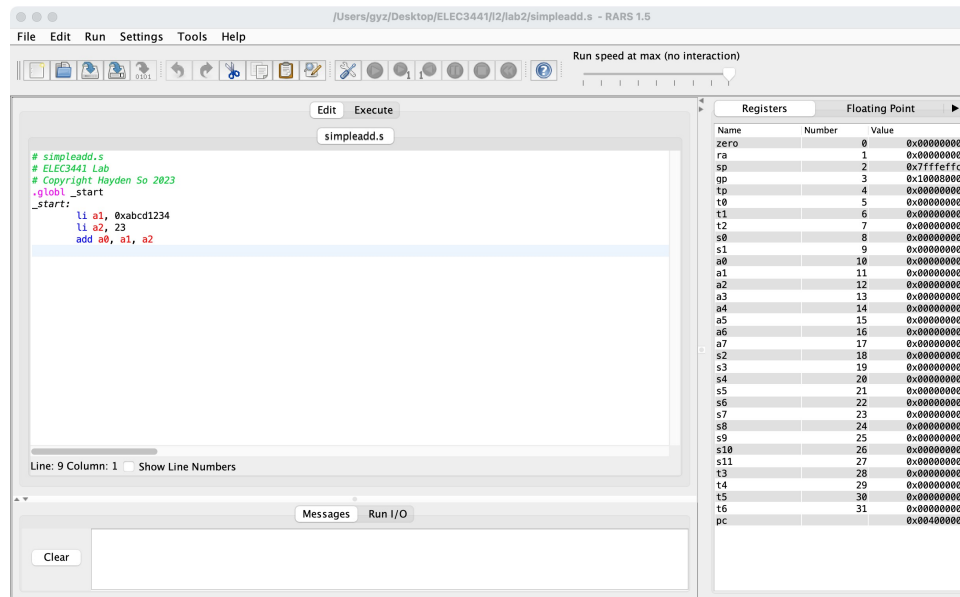
In the terminal inside the course container, run:

```
rars &
```

Open the browser-desktop address printed by `./manage up`. RARS should appear in that desktop. The Java runtime and the course's pinned RARS version are already installed in the container; do not download a separate JAR or Java installation.

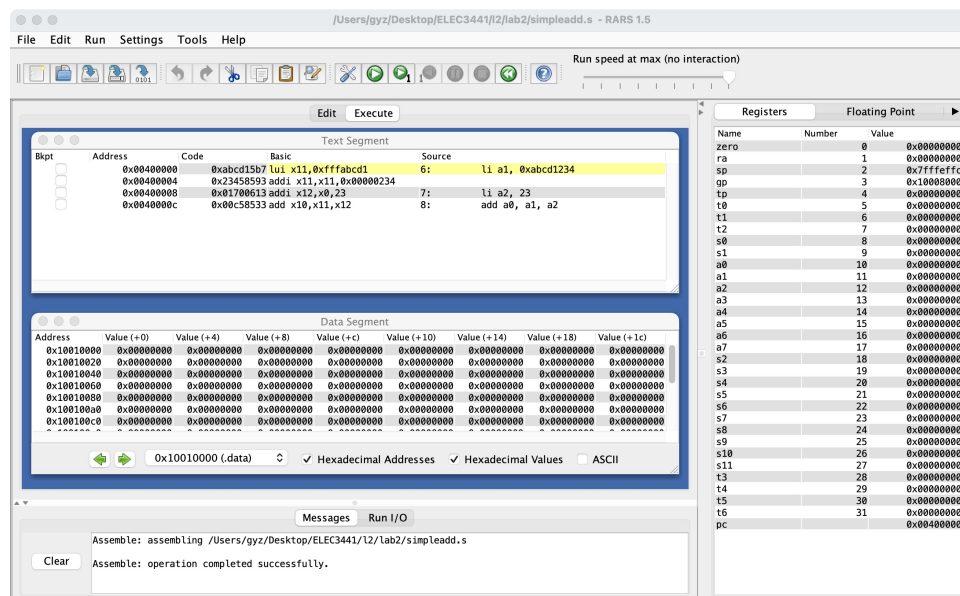
3 Your First Program

3.1 Opening `simpleadd.s` You should see the initial RARS interface, similar to the following:



Use the file-open button () at the top left to load `/workspace/labs/lab2/simpleadd.s`. Its contents should appear in the main **Edit** pane.

3.2 Running the Code To run the code, click the Assemble () button. A new **Execute** pane will show up that shows the current running program:



Within the **Execute** pane, two windows correspond to the **Text Segment** and the **Data Segment** of memory. Program instructions appear in the Text Segment, while program data are stored in the Data Segment.

3.3 Tracing Program Execution Now that the program is assembled and loaded into the main memory, you may execute the program in the assembler. You can run the whole program to the end with the **Run** button (). Alternatively, and most often, you will want to step through the program one instruction at a time by using the Step function (.

Now, step through the program one instruction at a time. After executing each line, make sure you understand how the register values are affected, including the PC register.

3.4 Check Yourself

Run the program and make sure you know the answer to these questions:

- (i) What value is stored in register `a0` when the program is first loaded?
- (ii) What is the value of `a0` after the program finishes running?
- (iii) What is the hardware register number of `a1` (0 to 31)?
- (iv) What is the memory address of the first instruction in your program?

4 Basic RISC-V Assembly Programming

4.1 Pseudo-instruction Load `/workspace/labs/lab2/li.s` and assemble the file.

4.2 Load Immediate Lines 6 and 7 of `li.s` use the load-immediate (`li`) pseudo-instruction. The `li` instruction loads a constant value into the target register `rd`:

```
li rd, <value>
```

Pseudo-instructions resemble the machine instructions described in class but are conveniences provided by the assembler. Before execution, each pseudo-instruction is translated into one or more machine instructions.

4.3 Examine the content of the **Text Segment** to see how the original `li` instructions are being assembled.

Consider line 6. When the constant fits in a signed 12-bit immediate, the assembler translates `li` into one `addi` machine instruction (`addi x11, x0, 23`). Register `x0` always contains zero, so the result is `x11=0+23=23`.

4.4 Larger constant If you try to load a constant outside the signed 12-bit range, the assembler will translate the `li` instruction differently.

4.5 Checkoff 1

Trace through the execution of the code and compare the machine instructions and their corresponding source assembly instructions. Then answer the following:

- (i) How many machine instructions does each of the two `li` instructions assemble into?
- (ii) Experiment with loading different constants. How are the `li` instructions being assembled differently with different constant values?

4.6 Jump Load the file `jump.s`.

4.7 `jump`, or `j`, is another commonly used pseudo instruction that jumps to a label (similar to `goto` in C language):

```
j <label>
```

This pseudo-instruction is usually translated into `jal` (*jump-and-link*) machine instruction as:

```
jal x0, <offset>
```

where *offset* is the address difference between the current PC and the `<label>` target. Because writing to `x0` has no effect, the return address is discarded.

4.8 Check Yourself

1. Examine the file. How are the branches and jump instructions translated?
2. Trace through the code and make sure you understand how the code is executed with the branch and jump instructions.
3. Consider the offset in the jump *machine* instruction, try to see if you can compute that offset from the relative difference between the jump instruction and the target label.

4.9 Load/Store Load the file `loadstore.s` into RARS.

4.10 RISC-V uses dedicated load and store instructions to transfer values between registers and memory.

```
l{b|h|w} rd, offset(rs1)
s{b|h|w} rs2, offset(rs1)
```

`b|h|w` refers to *byte*, *half-word*, and *word*. In a load or store instruction, register `rs1` holds the base address. Adding the immediate offset to that base gives the effective memory address.

4.11 Assemble the file `loadstore.s`. The program loads data stored in the label `arrayPrime`. The address of this label is hard coded in the file. You will explore how the address can be found automatically later in this lab.

4.12 Checkoff 2

Trace through the execution of the file `loadstore.s` and answer the following questions:

1. What are the values in the array `arrayPrime` after the code has completed?
2. What is the base address of `arrayPrime`?
3. If you insert another `.word` *before* the label `arrayPrime` and assemble the file again, where would the values of `arrayPrime` be moved to in memory?

5 Mysterious Program

Load `/workspace/labs/lab2/mystery.s` into RARS. It contains a complete program with a mysterious function and several assembler directives that may be unfamiliar. Follow the steps below to learn how they are used.

5.1 Text vs Data In `mystery.s`, you will notice two new directives: `.text` and `.data`.

The directive `.text` tells RARS to place the following **program instructions** in the *Text Segment* of main memory.

The directive `.data` tells RARS that the following lines should be loaded into the *Data Segment* of main memory.

In real systems, the operating-system kernel designates areas of main memory for different purposes. Some contain instructions, while others contain data. Common regions include text, data, and stack segments.

For this program, we will simply use the data and text segments.

5.2 The data labels `anArray` and `anArrayLen` are defined near the end of the file. Each `.word` directive reserves a 32-bit word and gives its initial value. In other words, `anArray` consists of six words, and the next word in memory stores `anArrayLen`.

Now **Assemble** the file and look into the **Execute** pane. Look in the **Data** segment. Can you find the data being loaded there? What is the actual address that `anArray` is stored at? What is the address of `anArrayLen`?

5.3 Look at line 9 and line 10 of the original `mystery.s`. There is a new pseudo instruction `la`, which has the format:

$$\text{la } \langle \text{rd} \rangle, \langle \text{label} \rangle$$

The function of the load the address instruction (`la`) is to load the memory address of the given label into the target register `rd`.

When writing position-independent code (PIC), the programmer does not encode a fixed location for the data. The program instead calculates an address relative to the current PC. Pseudo-instructions such as `la` let the assembler generate the required address-calculation instructions.

Now look in the Execute pane. In this RARS version, the `la` pseudo-instruction is translated into a two-instruction sequence using `auipc` followed by `addi`.

The `auipc` instruction places its 20-bit immediate in bits 31–12, adds that signed offset to the address of the `auipc` instruction, and writes the full sum to `rd`:

$$\text{rd} = \text{pc} + (\text{imm} \ll 12)$$

5.4 Look in the Execute pane. What machine instructions are produced from the two load address instructions in line 9 and 10? Why are they produced that way?

Hint: They are meant to find the address of the array `anArray` and the variable `anArrayLen`. You can look for them in the Data Segment and find out their addresses.

While tracing the loop, notice how `s2` is computed from the array base address and its length in bytes. Register `s2` holds the *one-past-the-end* address used as the loop sentinel. It is not the address of the last element and is never dereferenced.

5.5 Now trace through the code. How many times is the function `chk_number` called? Into which instructions is the `call` pseudo-instruction translated? Which instruction implements the `ret` pseudo-instruction?

5.6 `ecall` The `ecall` instructions invoke environment calls (also called system calls), which provide services to the program. In this lab, RARS provides the environment. It supports services such as `PrintInt` and `PrintString`; register `a7` selects the service, while registers `a0–a6` hold its arguments. See <https://github.com/TheThirdOne/rars/wiki/Environment-Calls> for details.

The `mystery.s` program uses `ecall` to print its final result in the **Run I/O** pane.

5.7 Checkoff 3

- (i) What is the value printed at the **Run I/O** pane after running the program?
- (ii) In words, describe what the program `mystery.s` does.

5.8 Submission Submit your answers to Checkoff 1, 2, and 3 above.